

动手学大模型应用开发 · 第 5 节

MCP 与 Skills 能力扩展

让工具可连接，让 Skills 可复用

主讲：陈思州



理论主线 · 先分工，再连接，最后组合

外部能力和Skill 指引解决的是不同问题，只有职责清楚，系统才容易替换和治理。

01 四层边界

调用表达、真实执行、协议连接、Skill 指引

02 MCP 连接

角色、能力发现、调用与返回

03 Skills 加载

目录、触发、正文与按需资源

04 组合与治理

选择、权限、证据、失败与评估

先回答“谁负责什么”，再讨论具体 SDK 和代码实现。

能力扩展包含四个不同层次

责任栈 · 从模型意图到可复用 Skill

同一次 Agent 行动可能同时经过四层，但每层的输入、输出和失败方式不同。

Agent 运行程序

选择能力、校验行动、执行并保存观察

01 Function Calling

模型提出名称与结构化参数

02 工具实现

程序执行真实读写、查询或计算

03 MCP

跨进程或网络发现并调用能力

04 Skills

按任务加载 Skill 正文与资源

模型提出调用不等于工具已经执行；激活 Skill 也不等于获得新的系统权限。

工具契约 · 先让能力可理解，再谈协议接入

同一个真实能力，如果描述、参数或返回语义含糊，模型就无法可靠选择和复查。

可调用工具契约

模型可理解、运行程序可校验、结果可追溯

01 名称与描述

说明能力、适用场景和不应调用的边界

02 参数结构

保留类型、必填项、枚举、格式与原始值

03 返回语义

区分结果、状态、来源和对下一步有用的字段

04 错误边界

区分参数、权限、环境与业务失败，并给出可处理信息

协议只负责传输契约；契约本身仍要由工具设计者写清。

协议架构 · 协调、连接与能力提供分离

一个 Host 可以连接多个 Server，但每条 Client 连接仍保持清晰的隔离边界。



Server 不应看到完整对话，也不能越过 Host 直接读取其他 Server 的上下文。

Host 通过独立 Client 隔离多个 Server

隔离边界 · 多服务由 Host 协调

一个 Host 可管理多个连接，但不让 Server 直接共享完整对话或彼此状态。



隔离由 Host 和部署边界实现；协议连通不代表默认共享。

消息路径 · 每次请求都能独立说明自己怎样通信

协议版本会演进，课程强调责任与错误处理，不把旧版握手当成永久规则。

01	声明元数据	版本、身份与客户端能力
02	发现能力	可先查询版本与能力
03	列出可用项	按能力读取工具、资源或提示
04	发起请求	携带名称、参数与 JSON-RPC id
05	处理结果	完整结果、需要输入或结构化错误

版本、能力与错误都应显式处理；协议成功不等于任务结果正确。

MCP 存在握手版与无状态版两种生命周期

A

2025-11-25 及更早

先完成 initialize 与 initialized，会话内保留协议版本、能力与会话识别。

边界

B

2026-07-28

取消协议握手和会话，可先用 server/discover 发现，每个请求自带版本与能力。

Tools、Resources 与 Prompts 承担不同职责

Server 原语 • 先判断需求是数据、动作还是模板

三类原语可以同时出现，但它们的控制方式、参数和结果语义不同。

Resources

可定位和读取的数据、文档或资源内容

1

Prompts

由用户或应用选择的参数化提示模板

3

2

Tools

模型可提出调用的查询、计算或外部动作

不要把所有能力都包装成工具；先根据交互责任选择合适原语。

Tools、Resources 与 Prompts 的控制方向不同

原语边界 · 同一 Server 可以同时暴露三类能力

先确定谁发起、谁选择、什么时候进入模型上下文。

Host

根据用户意图、应用策略和权限统一管理三类原语

01 Tools

模型可根据上下文提出 tools/call

02 Resources

应用决定列出、读取、订阅或装入哪些资料

03 Prompts

用户或应用选择模板并提供参数

04 可见性

三类都可以按授权、任务和当前能力筛选

先分清数据、行动与模板，再决定用哪类原语暴露。

A

stdio

Host 启动本地子进程，通过标准输入输出交换消息；部署简单，进程边界清晰。

边界

B

Streamable HTTP

通过网络端点连接远程服务；需要处理来源、认证、授权、网络失败与运维。

发现能力不等于允许执行

信任边界 · 能看见、能提出与能执行是三件事

协议负责互操作，Host 仍然负责授权、用户同意和最终执行边界。

01	发现	Server 声明当前可用能力
02	筛选	Host 按身份、任务和风险过滤
03	暴露	只把本轮需要的能力交给模型
04	校验	检查参数、权限、预算和确认
05	执行记录	保存调用、结果、错误和副作用

MCP 不是权限系统或沙箱；安全控制必须由 Host、Server 与部署环境共同实现。

一次工具调用要经历五个可核验状态

调用闭环 · 每一步都有独立失败点

把发现、选择、授权、执行与结果解释分开记录。

01	tools/list	获取当前授权下可见的工具与 Schema
02	选择候选	只把与当前任务有关的能力暴露给模型
03	执行前校验	检查类型、参数、权限、预算与用户确认
04	tools/call	用唯一请求 ID 发送名称与原始参数
05	结果处理	区分协议错误、isError 与可验证的正常结果

调用成功只证明返回了结果，不证明结论已经正确。

工具结果既要便于模型阅读，也要便于程序校验

结果契约 · 一份结果同时服务推理与程序处理

自然语言用于解释，结构化字段用于校验、路由、存储与自动化。

ToolResult

显式区分可读内容、结构化数据、错误状态与来源

01 content

给模型和用户的文本、图像、音频或资源

02 structuredContent

给程序的 JSON 数据，便于路由和保存

03 outputSchema

定义预期输出结构，客户端可以按此验证

04 状态与来源

保留 isError、请求 ID、来源、时间与截断边界

结果可读与结果可验证是两个独立要求。

工具结果进入模型前仍需校验

不可信输入 • 协议保证交换格式，不保证内容真实或安全

工具内容可以支持决策，但不应覆盖用户意图、系统约束或执行权限。

识别来源

记录 Server、工具、资源
URI、版本与请求 ID

1

校验范围

检查 Schema、数量、路
径、URI、MIME 与允许的任
务范围

3

分离数据与指令

外部内容作为证据，不提升
为高优先级命令

2

安全处理

接受、截断、隔离、重试或
请用户确认

4

可连接不等于可信；返回结果仍是需要处理的外部输入。

远程 MCP 授权要同时约束范围、资源与凭证

授权边界 · 协议连接之外仍要建立最小信任

HTTP 授权保护远程资源；Host 还要负责用户同意和最终执行决策。

远程 MCP 请求

每次请求都校验当前资源、身份、Scope 和操作意图

01 最小 Scope

初始只请求当前操作需要的权限

02 目标资源

令牌必须明确签发给将要访问的 MCP Server

03 凭证隔离

不跨发行方复用，不向下游 API 透传不匹配令牌

04 用户同意

展示请求方、操作、权限与影响，允许拒绝

认证身份、授权范围与用户同意是三个分开的检查。

Agent Skill 是包含 SKILL.md 的目录

Agent Skill • 用 SKILL.md 组织适用范围、步骤和资源

Skill 的结构服务于可发现、可加载和可维护，而不是把所有说明塞进一个超长提示。

SKILL.md

必需的元数据与任务说明

01 名称与描述

说明是什么，以及何时应该使用

02 正文

步骤、示例、边界与检查方法

03 scripts/

可执行且自包含的辅助代码

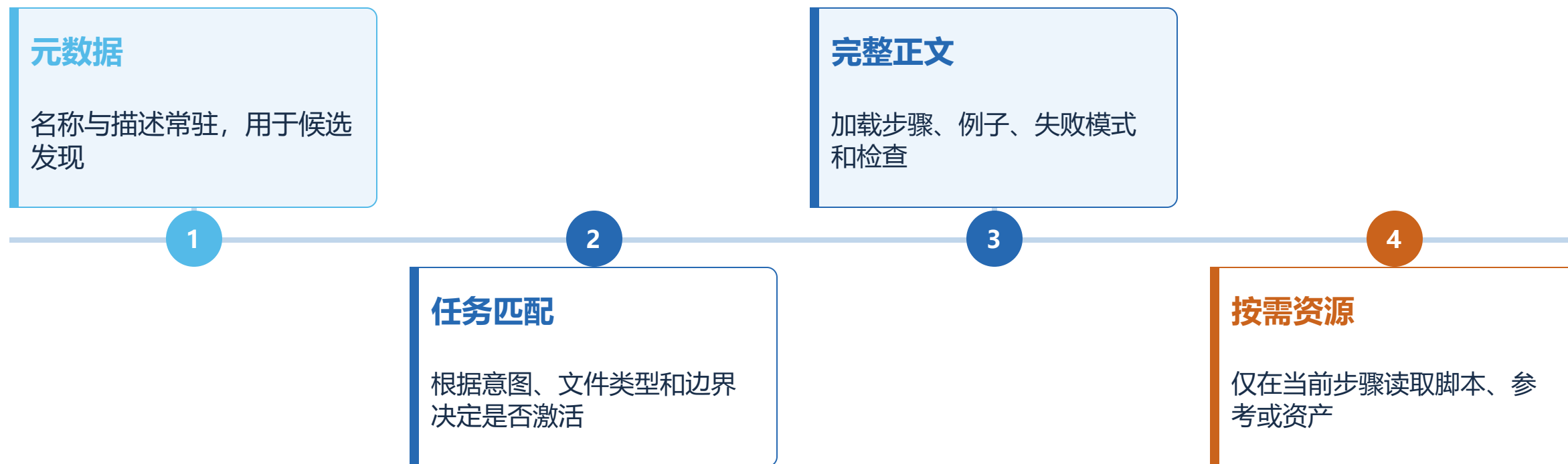
04 参考与资产

按需读取的资料、模板和静态资源

Skill 组织任务指引与资源；真实读写仍必须通过运行环境已经允许的工具。

加载顺序 · 从低成本发现到按需细节

每一层只在任务需要时进入上下文，既控制 Token，也保留可复用的深度资料。



描述决定能否被正确选中，正文和资源决定激活以后能否把任务做对。

主动发现 · 工具目录与真实执行分层

减少常驻 Schema 数量，同时保留授权、参数校验和失败处理。

- 01 浏览能力目录 只读取名称、用途、领域和适用边界等轻量元数据
- 02 按任务筛选 根据当前目标与环境挑选少量候选能力
- 03 加载完整契约 读取参数结构、返回语义、示例和限制
- 04 授权并执行 运行程序校验权限与参数，保存结果和调用记录

按需发现降低上下文负担，但不能绕过执行时的安全边界。

A

方法边界

SKILL.md 说明何时使用、执行步骤、资源位置、失败处理和验收方式。

边界

B

执行边界

Host 和工具层决定能读哪些文件、能否运行脚本、是否联网以及哪些动作需要确认。

A

工具设计

名称和参数无歧义，返回高信号且可操作的结果，错误信息能指导下一步。

边界

B

Skill 设计

描述写清何时使用与何时不用，正文给出步骤、例子、边界和验收方法。

A

MCP

解决应用怎样发现和调用外部能力，重点是协议、连接、输入结构、结果与权限边界。

边界

B

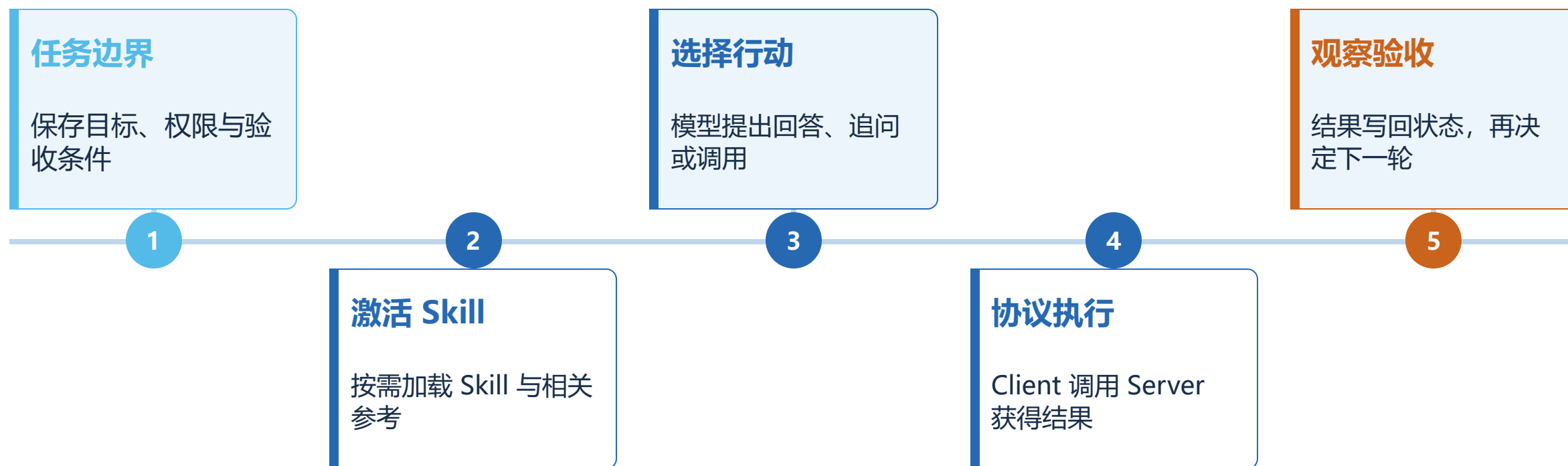
Skills

解决Skill 怎样被发现、激活和按需加载，重点是描述、步骤、资源与适用范围。

MCP 与 Skills 进入同一循环

联合运行 • Skill 按需加载，工具经 MCP 执行

两条路径在 Host 中汇合，但 Skill 加载不绕过权限，工具返回也不能覆盖系统规则。



Skill 指引、工具能力和运行控制分离，才能替换模型、Server 或 Skill 而不混淆责任。

一次可靠调用要留下完整证据链

审计轨迹 · 从任务意图追到最终结论

“调用成功”只证明通信完成，不证明参数正确、数据可信或任务已经满足。

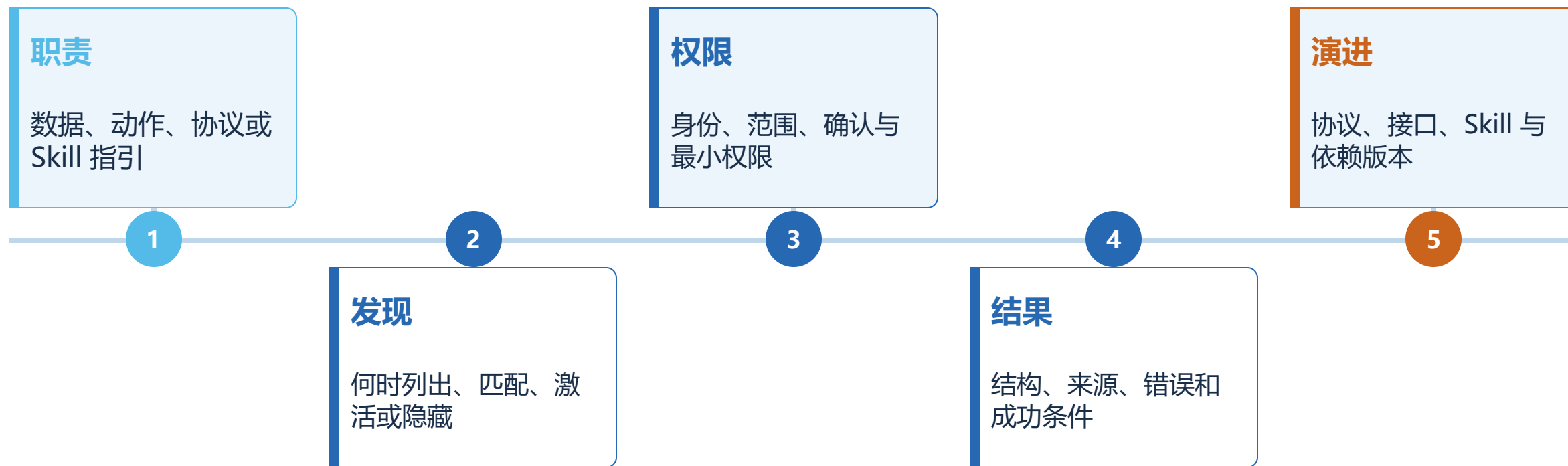
01	授权范围	用户目标、范围、风险与确认
02	候选能力	工具清单、Skill 名称与版本
03	调用记录	名称、参数、标识、耗时和错误
04	原始结果	保留结构、来源与未处理内容
05	结论核验	对照成功条件和原始证据检查

可追溯性让失败可以定位，也让协议或模型升级后的差异能够解释。

设计能力扩展时检查五个问题

设计检查 · 扩展能力之前先固定边界和证据

连接越多，选择歧义、权限风险和版本漂移越需要被显式管理。



先让责任和失败可解释，再追求更多工具或更自动的选择。

知识结构 · MCP 连接与 Skills 加载相互配合

MCP 解决怎样接入，Skills 解决怎样做，Host 负责把两者放进受控循环。



四层边界

表达、执行、协议与
Skill 指引



MCP 连接

角色、原语、调用、
权限与版本



Skills 加载

元数据、正文与按需
资源



组合治理

选择、授权、审计与
任务验收

让外部能力可互操作，让任务方法可复用，同时保留运行控制与证据边界。

本节关于 MCP 架构、生命周期、原语、授权安全与 Skills 规范主要参考以下官方资料。

[1]	Model Context Protocol	Architecture modelcontextprotocol.io/specification/2026-07-28/architecture
[2]	Model Context Protocol	The 2026-07-28 Specification blog.modelcontextprotocol.io/posts/2026-07-28/
[3]	Model Context Protocol	Tools modelcontextprotocol.io/specification/2026-07-28/server/tools
[4]	Model Context Protocol	Resources and Prompts modelcontextprotocol.io/specification/2026-07-28/server/
[5]	Model Context Protocol	Authorization modelcontextprotocol.io/specification/2026-07-28/basic/authorization
[6]	Model Context Protocol	Security Best Practices modelcontextprotocol.io/docs/2026-07-28/tutorials/security/security_best_practices
[7]	Agent Skills	Agent Skills Specification agentskills.io/specification
[8]	Anthropic	Writing Effective Tools for Agents anthropic.com/engineering/writing-tools-for-agents